# BUDGETED EMBEDDING TABLE FOR RECOMMENDER SYSTEMS

Yunke Qu, Quoc Viet Hung Nguyen, Tong Chen, Hongzhi Yin

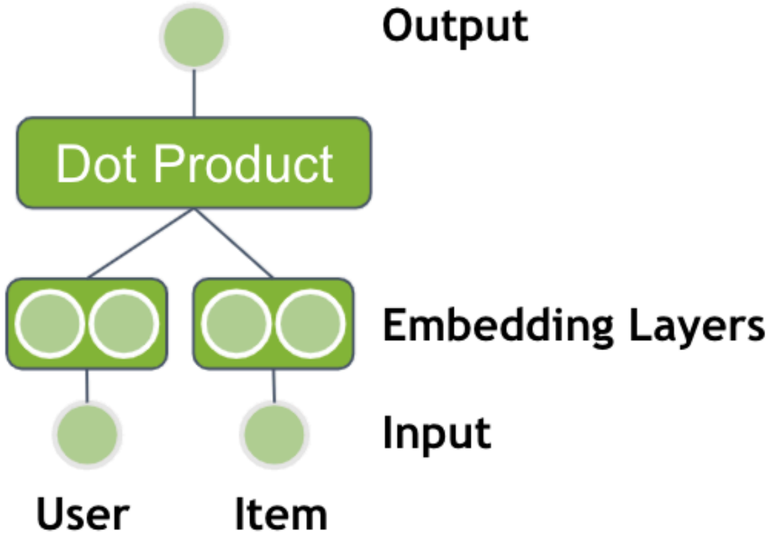
Grupo 4: Adriana Bastías, Andrés Mundaca, Andrés Riquleme

# Contexto

Los sistemas de recomendación modernos usan modelos de factores latentes con vectores de embedding para representar usuarios y elementos. Sin embargo, al aumentar la cantidad de usuarios y contenidos, este enfoque se vuelve ineficiente y poco escalable.

**Restricción implícita de memoria**

**Decisión instancia por instancia**



# Estado del Arte

- **Modelos basados en MLP**: utilizan redes profundas para aprender relaciones no lineales entre usuarios y elementos. Ej: NCF (Xiangnan He et al., 2020) , AutoRec(Suvash Sedhain et al., 2015) , Dual Contrastive Network (Suhang Wang et al., 2017)
- **Modelos basados en grafos**: difunden información a través del grafo para capturar relaciones indirectas y contextuales. Ej: NGCF(Xiang Wang et al., 2019), LightGCN (Xiangnan He et al., 2020)
- **Modelos basados en Factorization Machines**: combinan regresión lineal y factorización de matrices para modelar interacciones de segundo y mayor orden entre características. Ej: FM (Steffen Rendle., 2010)

**Limitaciones**

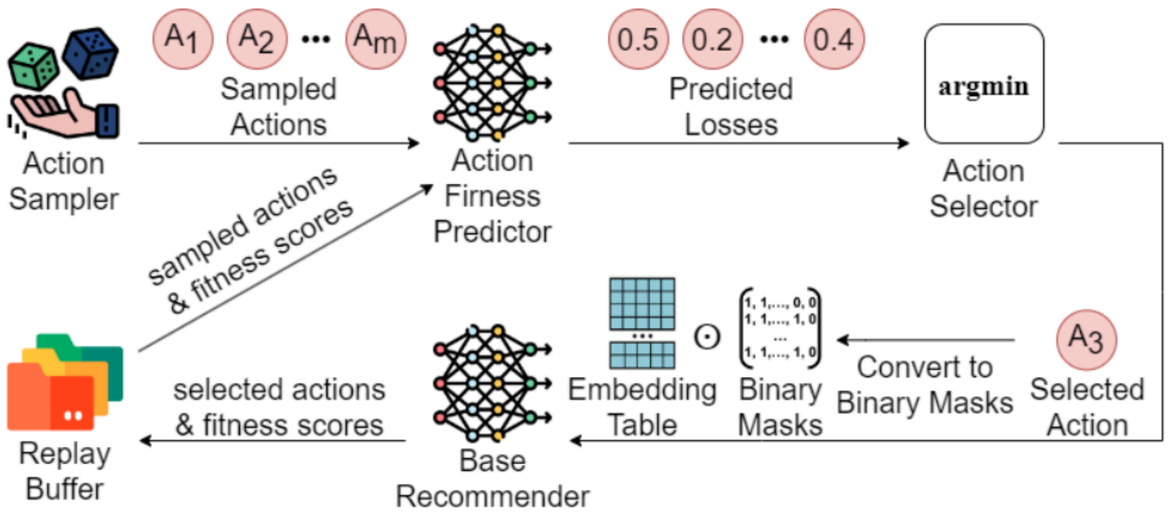
- **Alta complejidad en entrenamiento**
- **Alto costo computacional**

# Problema de Recomendación

Mejorar cómo se asignan los tamaños de los embeddings en sistemas de recomendación bajo restricciones de memoria específica

## Budgeted Embedding Table

Un paradigma de búsqueda automatizada del tamaño de embeddings de entrada.



# Contribución

| | | |
|---|---|---|
| 01 | **Identificación de cuellos de botella en los métodos actuales** | Parámetros no escalables e ineficiencia en tamaños de embedding para cada instancia. |
| 02 | **Propuesta de BET** | Realiza la reducción de dimensión en un solo paso (table-level action) y una red basada en DeepSets. |
| 03 | **Validación experimental en dos conjuntos de datos reales** | Confirma la eficacia de BET. |

# Marco Teórico

## BET

Budgeted Embedding Table

## Embedding - Embedding Table

Representación numérica de un elemento en forma de vector. La colección de todos estos vectores se guarda en una Embedding Table.

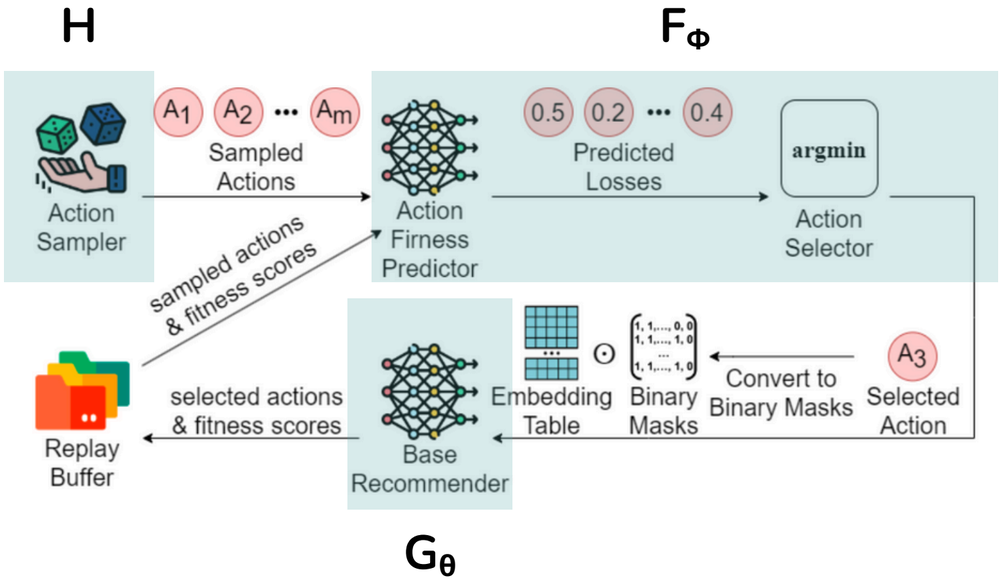
## DeepSet

Técnica de aprendizaje que permite representar conjuntos de elementos de manera que el modelo sea invariante al orden.

# Descripción de BET

Tiene 3 componentes principales:

- **$G_\theta$:** Recomendador *backbone* con una tabla de embedding *full-size, parametrizado por $\theta$*
- **$F_\Phi$:** *"Action Fitness Prediction Network"*, parametrizada por $\Phi$
- **H:** *Action Sampler no parametrico*



# Embedding Masking

- $\mathbf{G_\theta}$ base con tabla de embeddings $\mathbf{E}$ de tamaño $(|U|+|V|)\times d_{max}$
- Cada usuario/item tiene un tamaño de embedding variable $\mathbf{d_n}$
- Se introduce la mascara binaria $M \in \{0,1\}^{(|U|+|V|)\times dmax}$.
  - Activa las primeras $\mathbf{d_n}$ dimensiones y desactiva el resto
  - Permite ajustar la representación según relevancia
- En deploy, solo se guarda $E_{sparse} = E \odot M$, reduciendo el costo de memoria

# Embedding Masking: Formulación

- Embedding lookup con máscara: $\mathbf{e}_n = (\mathbf{E} \odot \mathbf{M})[n]$
- Calculo de afinidad usuario-item: $\mathbf{y}_{uv} = G_{\Theta}(\mathbf{e}_u, \mathbf{e}_v)$, $u \in \mathbf{U}, v \in \mathbf{V}$
- Perdida BPR: $\mathcal{L}_{BPR} = \sum_{(u,v,v') \in \mathcal{D}_{train}} -\ln \sigma(\hat{y}_{uv} - \hat{y}_{uv'}) + \eta \|\Theta\|_2^2,$
- Minimización objetivo: $\min_{\Theta,\Phi} \mathcal{L}_{BPR} \quad \text{s.t. } 1 - \frac{\|\mathbf{M}\|_{1,1}}{(|\mathcal{U}| + |\mathcal{V}|) \times d_{max}} \leq c,$

  ,donde **c** es el target sparsity rate

# Budget Aware Action Sampling

- Metodo para generar una acción **a** en cada iteración

- Se definen las distribuciones $\mathbf{P_u}$ y **Pv** (ley de potencia, normal truncada o log normal), parametrizadas por $\beta_{\mathcal{U}}, \beta_{\mathcal{V}} \sim \mathrm{Uniform}(0, \beta_{max})$,

- Se obtienen las probabilidades normalizadas $\mathbf{p_i}$ y $\mathbf{p_j}$: representan la fracción de parametros de cada elemento

- Se calcula el tamaño del embedding con:

$$d_i = \lfloor \tilde{p}_i \cdot w \cdot d_{max} \cdot (|\mathcal{U}| + |\mathcal{V}|) \cdot c \rfloor, \quad i = 1, 2, \cdots, |\mathcal{U}|,$$
$$d_j = \lfloor \tilde{p}_j \cdot (1 - w) \cdot d_{max} \cdot (|\mathcal{U}| + |\mathcal{V}|) \cdot c \rfloor, \quad j = 1, 2, \cdots, |\mathcal{V}|,$$

donde **w**~Uniforme(0,1).

# Budget Aware Action Sampling

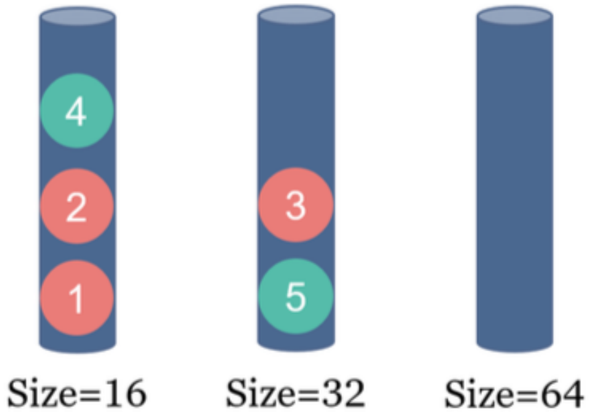
- Como los usuarios/items más frecuentes suelen ser más informativos, se ranquea el n-grande embedding size con el n-frecuente elemento.

- El resultado es que se generan **m** acciones candidatas $\mathbf{Q_t=\{a_1,\dots,a_m\}}$ para evaluar con la red de predicción en cada iteración

# Fitness Prediction Network $F_\Phi$

- Para elegir la mejor acción propuesta de $\mathbf{Q_t}$, se utiliza una **fitness prediction network** basada en los deep sets

- Se representan los usuarios en sets $a = \{\mathcal{S}_d\}_{d=1}^{d_{max}}$, con $\mathcal{S}_d = \{n | \forall n \in \mathcal{U} \cup \mathcal{V}, d_n = d\}$ donde cada $\mathbf{S_d}$ agrupa usuarios/items con el mismo tamaño de embedding




Visualización de los sets

# Arquitectura de $F_\Phi$

- Encoders de usuarios e items (utiliza frecuencia):

$$\boldsymbol{q}_u = \rho_{\mathcal{U}}\left(\frac{f_u}{\max_{u' \in \mathcal{U}} f_{u'}}\right), \quad u \in \mathcal{U},$$

$$\boldsymbol{q}_v = \rho_{\mathcal{V}}\left(\frac{f_v}{\max_{v' \in \mathcal{V}} f_{v'}}\right), \quad v \in \mathcal{V},$$

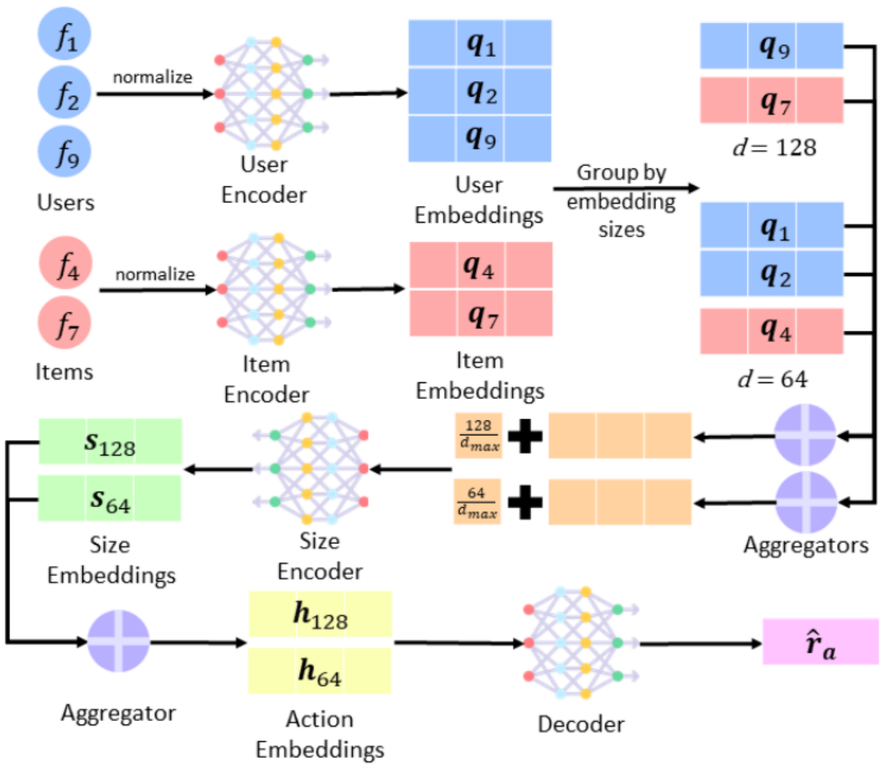
- Encoder de tamaño de embedding, con manejo de conjuntos:

$$\boldsymbol{s}_d = \mu\left(\left[\frac{1}{|\mathcal{S}_d|}\sum_{n \in \mathcal{S}_d} \boldsymbol{q}_n; \frac{d}{d_{max}}\right]\right),$$

- Decoder $\pi$ para obtener un predicted fitness score $\mathbf{r}_a$:

$$\boldsymbol{h}_a = \frac{1}{d_{max}}\sum_{d=1}^{d_{max}} \boldsymbol{s}_d, \quad \longrightarrow \quad \hat{r}_a = \pi(\boldsymbol{h}_a).$$

- Todos estos elementos con distnto MLP



# Fitness Predictor Training

- Se busca optimizar $\mathbf{F_\Phi}$ con tal de que prediga correctamente el $\mathbf{r_a}$ de cada acción **a.** En cada iteración:
  - Se genera un conjunto de acciones candidatas $\mathbf{Q_t=\{a_1,\dots,a_m\}}$
  - Se elige una acción prometedora $\mathbf{a_t}$.
  - Se ajusta el recomendador $G_\Theta$ usando $\mathbf{a_t}$.
  - Se evalúa el desempeño real de esa acción → $\mathbf{r_a}$.
  - Se entrena $\mathbf{F_\Phi}$ minimizando el error entre la predicción y el valor real.

- La optimización se hace minimizando MSE con gradient descent:

$$\underset{\Phi'}{\operatorname{argmin}} \sum_{\forall a} (r_a - \hat{r}_a)^2.$$

# Fitness Predictor Training

- El valor del fitness real se calcula con: a

$$r_a = \frac{\text{eval}(\mathbf{E} \odot \mathbf{M}_a | \mathcal{D}_{val})}{\text{eval}(\mathbf{E} | \mathcal{D}_{val})},$$

- Con $\mathbf{M_a}$ es la mascara binaria de la acción a y $\mathbf{D_{val}}$ es el validation set

- **eval()** es una mezcla de Recall@k y NDCG@k con k∈K{5,10,20}:

$$\text{eval}(\mathbf{E} \odot \mathbf{M} | \mathcal{D}_{val}) = \sum_{u \in \mathcal{U}} \frac{\sum_{k \in \mathcal{K}} \text{Recall@}k_u + \text{NDCG@}k_u}{2|\mathcal{K}||\mathcal{U}|},$$

- Esto logra:
  - Reducir **m x T** a solo **T** evaluaciones reales
  - Refinar el predictor con ejemplos nuevos en cada iteracion

# Flujo completo

- Líneas 1-2 → Inicialización:
  - Se inicializan los parámetros del recomendador $G_\Theta$ y el predictor $\mathbf{F}_\Phi$ Se entrena $G_\Theta$ hasta convergencia con BPR loss. Se crea el conjunto de acciones **A.**

- Líneas 3-12 → Bucle de búsqueda:
  - Se generan **m** acciones candidatas $\mathbf{Q_t=\{a_1,\dots,a_m\}}$

- Se selecciona una acción $\mathbf{a_t}$ con tres estrategias:
  - Greedy → elige la acción con mayor fitness predicho FΦ(a)
  - Random → explora nuevas distribuciones.
  - (líneas 10-11): Vecindad → acción más cercana al mejor embedding previo.

**Algorithm 1** Embedding Size Search with BET

1: Initialize $\Theta$ and $\Phi$, set $\mathcal{A} \leftarrow \emptyset$;
2: Train $G_\Theta(\cdot)$ till convergence w.r.t. Eq. 2;
3: **for** $t = 1, \cdots, T$ **do**
4: Draw distributions and their parameters w.r.t. Eq. 4;
5: Draw $m$ actions $Q_t = \{a_1, ..., a_m\}$ with Eq. 5 and Eq. 6;
6: **if** $0 \leq t \bmod 5 \leq 2$ **then**
7: $a_t \leftarrow$ Strategy I;
8: **else if** $t \bmod 5 = 3$ **then**
9: $a_t \leftarrow$ Strategy II;
10: **else if** $t \bmod 5 = 4$ **then**
11: $a_t \leftarrow$ Strategy III;
12: **end if**
13: Finetune $\Theta$ w.r.t. $\mathbf{E} \odot \mathbf{M}_{a_t}$ and Eq. 2
14: $r_{a_t} \leftarrow$ Eq. 13
15: $\mathcal{A} \leftarrow \mathcal{A} \cup (a_t, r_{a_t})$;
16: **for** $j = 1, \cdots, N$ **do**
17: Randomly sample $(a_j, r_{a_j})$ from $\mathcal{A}$;
18: $\hat{r}_{a_j} \leftarrow$ Eq. 10;
19: Update $\Phi$ w.r.t. Eq. 11;
20: **end for**
21: **end for**
22: Perform selective retraining and obtain $a^* \leftarrow$ Eq. 14.

# Flujo completo

- Líneas 13-15 → Evaluación real
  - Se ajusta el recomendador $G_\Theta$ con la acción elegida $\mathbf{a_t}$
  - Se calcula el fitness real $\mathbf{r_{at}}$
  - Se añade el par ($\mathbf{a_t}$ ,$\mathbf{r_{at}}$ ) al conjunto **A**

- Líneas 16-20 → Entrenamiento del predictor:
  - Se muestrean ejemplos de **A** y se actualiza $\mathbf{F_\Phi}$ minimizando el MSE
  - El predictor mejora su precisión iterativamente

**Algorithm 1** Embedding Size Search with BET

1: Initialize $\Theta$ and $\Phi$, set $\mathcal{A} \leftarrow \emptyset$;
2: Train $G_\Theta(\cdot)$ till convergence w.r.t. Eq. 2;
3: **for** $t = 1, \cdots, T$ **do**
4: Draw distributions and their parameters w.r.t. Eq. 4;
5: Draw $m$ actions $Q_t = \{a_1, ..., a_m\}$ with Eq. 5 and Eq. 6;
6: **if** $0 \leq t \bmod 5 \leq 2$ **then**
7: $a_t \leftarrow$ Strategy I;
8: **else if** $t \bmod 5 = 3$ **then**
9: $a_t \leftarrow$ Strategy II;
10: **else if** $t \bmod 5 = 4$ **then**
11: $a_t \leftarrow$ Strategy III;
12: **end if**
13: Finetune $\Theta$ w.r.t. $\mathbf{E} \odot \mathbf{M}_{a_t}$ and Eq. 2
14: $r_{a_t} \leftarrow$ Eq. 13
15: $\mathcal{A} \leftarrow \mathcal{A} \cup (a_t, r_{a_t})$;
16: **for** $j = 1, \cdots, N$ **do**
17: Randomly sample $(a_j, r_{a_j})$ from $\mathcal{A}$;
18: $\hat{r}_{a_j} \leftarrow$ Eq. 10;
19: Update $\Phi$ w.r.t. Eq. 11;
20: **end for**
21: **end for**
22: Perform selective retraining and obtain $a^* \leftarrow$ Eq. 14.

# Flujo completo

- Línea 22 → Retraining final:
  - Tras **T** iteraciones, se reentrena el modelo con las 5 mejores acciones (mayor $\mathbf{r_a}$)

$$a^* = \underset{a \in \mathcal{A}'}{\operatorname{argmax}}\ r_a,$$

- Resultado final: tabla de embeddings optimizada

$$\mathbf{E}_{\text{sparse}} = \mathbf{E} \odot \mathbf{M}_{a^*}$$

**Algorithm 1** Embedding Size Search with BET

1: Initialize $\Theta$ and $\Phi$, set $\mathcal{A} \leftarrow \emptyset$;
2: Train $G_\Theta(\cdot)$ till convergence w.r.t. Eq. 2;
3: **for** $t = 1, \cdots, T$ **do**
4: Draw distributions and their parameters w.r.t. Eq. 4;
5: Draw $m$ actions $\mathcal{Q}_t = \{a_1, ..., a_m\}$ with Eq. 5 and Eq. 6;
6: **if** $0 \leq t \bmod 5 \leq 2$ **then**
7: $a_t \leftarrow$ Strategy I;
8: **else if** $t \bmod 5 = 3$ **then**
9: $a_t \leftarrow$ Strategy II;
10: **else if** $t \bmod 5 = 4$ **then**
11: $a_t \leftarrow$ Strategy III;
12: **end if**
13: Finetune $\Theta$ w.r.t. $\mathbf{E} \odot \mathbf{M}_{a_t}$ and Eq. 2
14: $r_{a_t} \leftarrow$ Eq. 13
15: $\mathcal{A} \leftarrow \mathcal{A} \cup (a_t, r_{a_t})$;
16: **for** $j = 1, \cdots, N$ **do**
17: Randomly sample $(a_j, r_{a_j})$ from $\mathcal{A}$;
18: $\hat{r}_{a_j} \leftarrow$ Eq. 10;
19: Update $\Phi$ w.r.t. Eq. 11;
20: **end for**
21: **end for**
22: Perform selective retraining and obtain $a^* \leftarrow$ Eq. 14.

# Evaluación

Se compara BET con los siguientes modelos base:

- PEP. Pruning de embedding durante el entrenamiento
- ESAPN. Embedding según complejidad de los datos
- OptEmbed. Pruning de embedding con algoritmos de búsqueda
- CIESS. Dominio continuo en vez de discreto para embedding
- Equal Sizes de embeddings
- Mixed and Random embeddings

# Rendimiento de memoria

- Bajo el mismo presupuesto de memoria, BET tuvo mejor rendimiento
- Al reducir el presupuesto de memoria, BET pierde menos rendimiento
- Con presupuestos de memoria aumentados, BET se acerca al rendimiento de embeddings completos
- A diferencia de los otros métodos, BET garantiza no excederse del presupuesto de memoria

| | | LightGCN | | | |
|---|---|---|---|---|---|
| Method | Sparsity | R@5 | N@5 | R@20 | N@20 |
| ESAPN | 75% | 0.0741 | 0.1173 | 0.1579 | 0.1329 |
| OptEmb | 74% | 0.0591 | 0.0983 | 0.1314 | 0.1115 |
| PEP | 80% | 0.0715 | 0.1189 | 0.1588 | 0.1347 |
| ES | | 0.0756 | 0.1219 | 0.1674 | 0.1400 |
| MR | | 0.0708 | 0.1150 | 0.1583 | 0.1323 |
| CIESS | | 0.0777 | 0.1263 | 0.1750 | 0.1461 |
| BET | | **0.0797** | **0.1303** | **0.1753** | **0.1483** |
| *p*-value | | 0.0024 | 0.0001 | 0.0391 | 0.0012 |
| PEP | 90% | 0.0594 | 0.0992 | 0.1274 | 0.1102 |
| ES | | 0.0646 | 0.1041 | 0.1477 | 0.1217 |
| MR | | 0.0617 | 0.1013 | 0.1403 | 0.1171 |
| CIESS | | 0.0721 | 0.1193 | 0.1589 | 0.1353 |
| BET | | **0.0736** | **0.1212** | **0.1620** | **0.1376** |
| *p*-value | | 0.0165 | 0.0481 | 0.0091 | 0.0226 |
| PEP | 95% | 0.0372 | 0.0666 | 0.0803 | 0.0712 |
| ES | | 0.0446 | 0.0727 | 0.1085 | 0.0872 |
| MR | | 0.0500 | 0.0834 | 0.1178 | 0.0975 |
| CIESS | | 0.0513 | 0.0853 | 0.1214 | 0.0997 |
| BET | | **0.0627** | **0.1037** | **0.1374** | **0.1171** |
| *p*-value | | 0.0001 | 0.0002 | 0.0001 | 0.0001 |

# Eficiencia de optimización

Gracias a acciones a nivel de tabla, usando el fitness predictor se pueden ajustar los parámetros sin tener que reentrenar para todos los datos

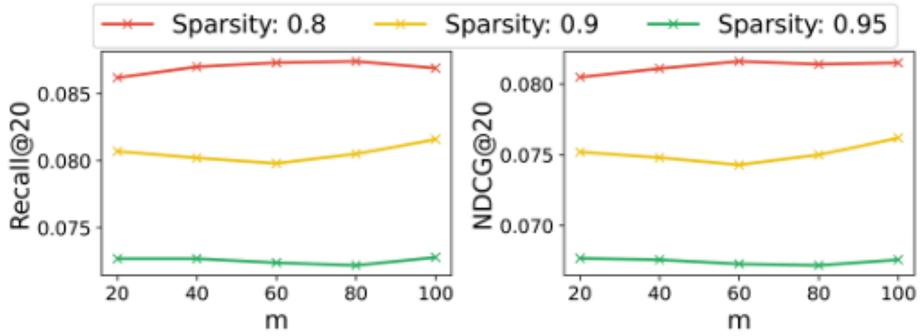
Donde distribuciones más dispersas de embedding tuvieron mejor rendimiento en general

| Sparsity | Selection Method | Yelp2018 | | Gowalla | |
|---|---|---|---|---|---|
| | | R@20 | N@20 | R@20 | N@20 |
| 80% | Random Selection | 0.0846 | 0.0788 | 0.1749 | 0.1473 |
| | Simple fitness prediction | 0.0835 | 0.0779 | 0.1739 | 0.1469 |
| | DeepSets fitness prediction | **0.0873** | **0.0816** | **0.1753** | **0.1483** |
| 90% | Random Selection | 0.0767 | 0.0717 | 0.1594 | 0.1357 |
| | Simple fitness prediction | 0.0768 | 0.0717 | 0.1590 | 0.1355 |
| | DeepSets fitness prediction | **0.0816** | **0.0762** | **0.1620** | **0.1376** |
| 95% | Random Selection | 0.0637 | 0.0583 | 0.1335 | 0.1143 |
| | Simple fitness prediction | 0.0673 | 0.0629 | 0.1349 | 0.1152 |
| | DeepSets fitness prediction | **0.0727** | **0.0676** | **0.1374** | **0.1171** |

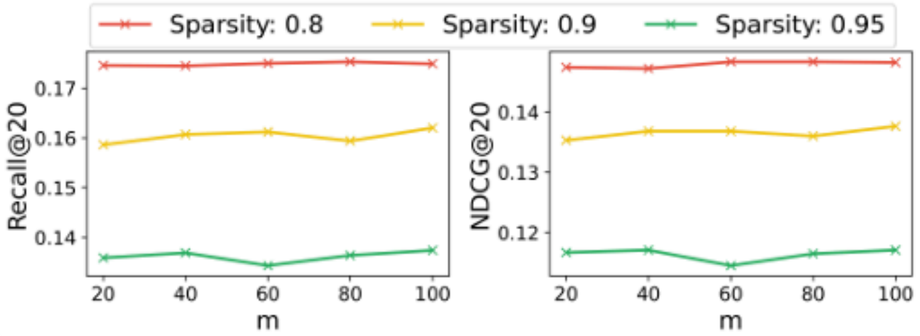
# Dimensión de embeddings

Al asignar dimensiones a distintos grupos del dataset (set representation), se le da más parte del presupuesto a los más relevantes, y menos al resto

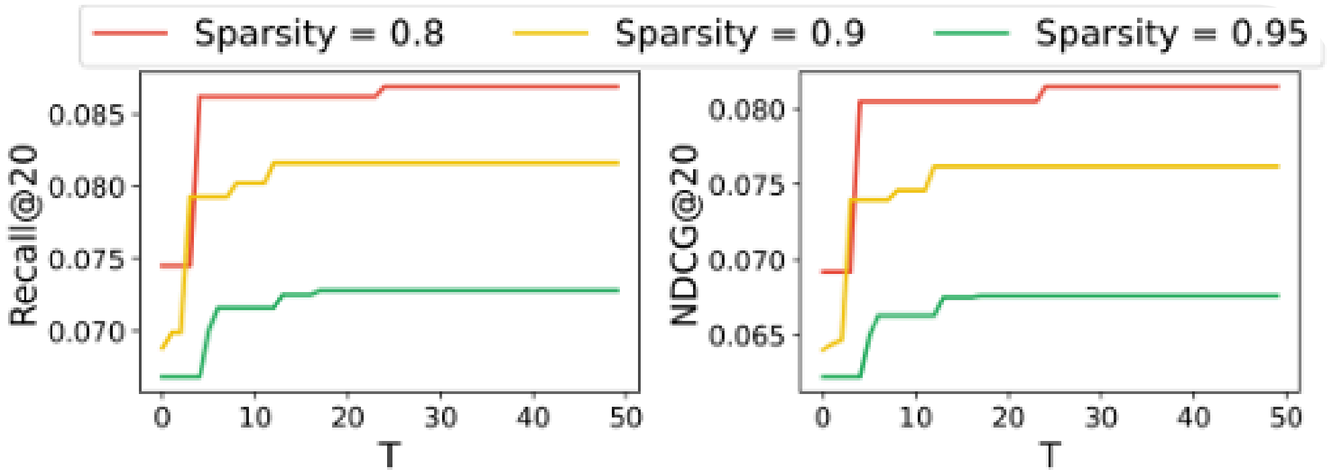
Particularmente, el parámetro T tiene el mayor impacto



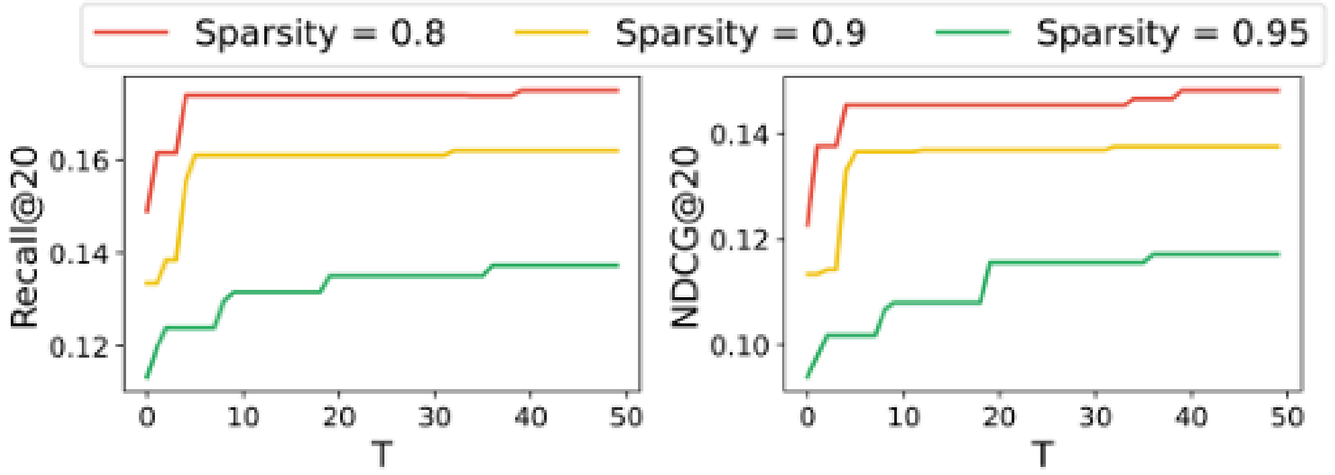

(a) Effect of $m$ on Yelp




(b) Effect of $m$ on Gowalla




(a) Effect of $T$ on Yelp




(b) Effect of $T$ on Gowalla

# Conclusión

El modelo BET, ofrece mejor rendimiento que todos los otros modelos estado del arte, si se comparan con un límite de memoria fijo. Sin perder información relevante para las recomendaciones

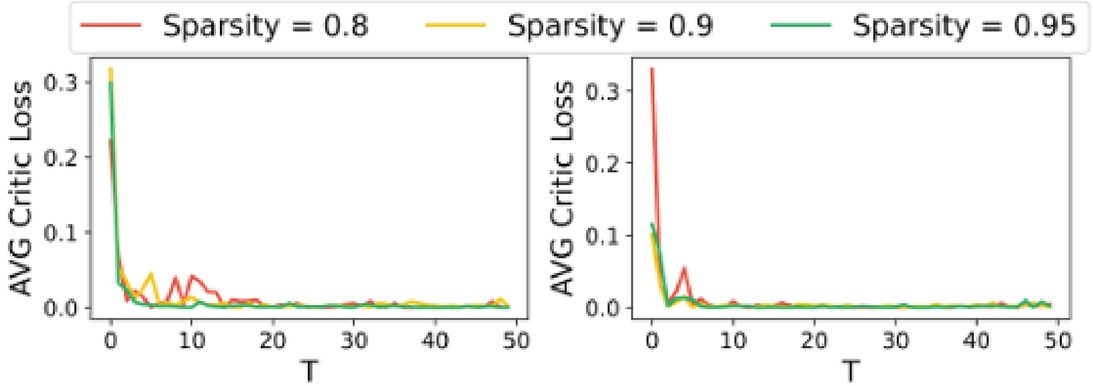



**Figure 6: Sensitivity analysis of $T$ on the fitness prediction loss with LightGCN as the backbone recommender on Yelp2018 (left) and Gowalla (right).**

# Referencias

Yunke Qu, et al. 2025. Budgeted Embedding Table For Recommender Systems

# Gowalla

| | LightGCN | | | | | NGCF | | | | | NCF | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Method | Sparsity | R@5 | N@5 | R@20 | N@20 | Sparsity | R@5 | N@5 | R@20 | N@20 | Sparsity | R@5 | N@5 | R@20 | N@20 |
| ESAPN | 75% | 0.0741 | 0.1173 | 0.1579 | 0.1329 | 83% | 0.0627 | 0.1060 | 0.1396 | 0.1193 | 74% | 0.0498 | 0.0718 | 0.1280 | 0.0959 |
| OptEmb | 74% | 0.0591 | 0.0983 | 0.1314 | 0.1115 | 75% | 0.0510 | 0.0860 | 0.1227 | 0.1005 | 82% | 0.0330 | 0.0492 | 0.0877 | 0.0653 |
| PEP | | 0.0715 | 0.1189 | 0.1588 | 0.1347 | | 0.0656 | 0.1049 | **0.1524** | 0.1245 | | 0.0498 | 0.0704 | 0.1282 | 0.0957 |
| ES | | 0.0756 | 0.1219 | 0.1674 | 0.1400 | | 0.0619 | 0.0961 | 0.1466 | 0.1170 | | 0.0481 | 0.0662 | 0.1235 | 0.0916 |
| MR | 80% | 0.0708 | 0.1150 | 0.1583 | 0.1323 | 80% | 0.0512 | 0.0815 | 0.1223 | 0.0982 | 80% | 0.0478 | 0.0656 | 0.1243 | 0.0910 |
| CIESS | | 0.0777 | 0.1263 | 0.1750 | 0.1461 | | **0.0676** | **0.1084** | 0.1504 | **0.1251** | | **0.0550** | **0.0744** | **0.1307** | **0.0997** |
| BET | | **0.0797** | **0.1303** | **0.1753** | **0.1483** | | 0.0643 | 0.1039 | 0.1454 | 0.1204 | | 0.0511 | 0.0723 | 0.1294 | 0.0973 |
| $p$-value | | 0.0024 | 0.0001 | 0.0391 | 0.0012 | | 0.7815 | 0.2179 | 0.0514 | 0.0924 | | 0.1962 | 0.1337 | 0.2578 | 0.0414 |
| PEP | | 0.0594 | 0.0992 | 0.1274 | 0.1102 | | 0.0552 | 0.0914 | 0.1327 | 0.1081 | | 0.0414 | 0.0631 | 0.1121 | 0.0839 |
| ES | | 0.0646 | 0.1041 | 0.1477 | 0.1217 | | 0.0589 | 0.0935 | 0.1380 | 0.1115 | | 0.0413 | 0.0548 | 0.1112 | 0.0796 |
| MR | 90% | 0.0617 | 0.1013 | 0.1403 | 0.1171 | 90% | 0.0466 | 0.0764 | 0.1123 | 0.0911 | 90% | 0.0399 | 0.0536 | 0.1069 | 0.0769 |
| CIESS | | 0.0721 | 0.1193 | 0.1589 | 0.1353 | | 0.0601 | 0.0980 | 0.1380 | 0.1140 | | **0.0448** | 0.0613 | **0.1131** | 0.0841 |
| BET | | **0.0736** | **0.1212** | **0.1620** | **0.1376** | | **0.0623** | **0.1012** | **0.1426** | **0.1177** | | 0.0434 | **0.0648** | 0.1122 | **0.0845** |
| $p$-value | | 0.0165 | 0.0481 | 0.0091 | 0.0226 | | 0.0005 | 0.0205 | 0.0601 | 0.0232 | | 0.1454 | 0.0554 | 0.6608 | 0.0655 |
| PEP | | 0.0372 | 0.0666 | 0.0803 | 0.0712 | | 0.0247 | 0.0471 | 0.0674 | 0.0554 | | 0.0351 | 0.0591 | 0.0897 | 0.0712 |
| ES | | 0.0446 | 0.0727 | 0.1085 | 0.0872 | | 0.0527 | 0.0870 | **0.1213** | **0.1007** | | 0.0336 | 0.0478 | 0.0952 | 0.0682 |
| MR | 95% | 0.0500 | 0.0834 | 0.1178 | 0.0975 | 95% | 0.0438 | 0.0739 | 0.1040 | 0.0859 | 95% | 0.0329 | 0.0489 | 0.0918 | 0.0674 |
| CIESS | | 0.0513 | 0.0853 | 0.1214 | 0.0997 | | 0.0505 | 0.0856 | 0.1129 | 0.0964 | | 0.0361 | 0.0567 | 0.0933 | 0.0717 |
| BET | | **0.0627** | **0.1037** | **0.1374** | **0.1171** | | **0.0528** | **0.0883** | 0.1172 | 0.9980 | | **0.0388** | **0.0616** | **0.0963** | **0.0756** |
| $p$-value | | 0.0001 | 0.0002 | 0.0001 | 0.0001 | | 0.1896 | 0.0638 | 0.7269 | 0.3608 | | 0.0321 | 0.1107 | 0.1104 | 0.0010 |

# Yelp

| | LightGCN | | | | | NGCF | | | | | NCF | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Method | Sparsity | R@5 | N@5 | R@20 | N@20 | Sparsity | R@5 | N@5 | R@20 | N@20 | Sparsity | R@5 | N@5 | R@20 | N@20 |
| ESAPN | 75% | 0.0741 | 0.1173 | 0.1579 | 0.1329 | 83% | 0.0627 | 0.1060 | 0.1396 | 0.1193 | 74% | 0.0498 | 0.0718 | 0.1280 | 0.0959 |
| OptEmb | 74% | 0.0591 | 0.0983 | 0.1314 | 0.1115 | 75% | 0.0510 | 0.0860 | 0.1227 | 0.1005 | 82% | 0.0330 | 0.0492 | 0.0877 | 0.0653 |
| PEP | | 0.0715 | 0.1189 | 0.1588 | 0.1347 | | 0.0656 | 0.1049 | **0.1524** | 0.1245 | | 0.0498 | 0.0704 | 0.1282 | 0.0957 |
| ES | | 0.0756 | 0.1219 | 0.1674 | 0.1400 | | 0.0619 | 0.0961 | 0.1466 | 0.1170 | | 0.0481 | 0.0662 | 0.1235 | 0.0916 |
| MR | 80% | 0.0708 | 0.1150 | 0.1583 | 0.1323 | 80% | 0.0512 | 0.0815 | 0.1223 | 0.0982 | 80% | 0.0478 | 0.0656 | 0.1243 | 0.0910 |
| CIESS | | 0.0777 | 0.1263 | 0.1750 | 0.1461 | | **0.0676** | **0.1084** | 0.1504 | **0.1251** | | **0.0550** | **0.0744** | **0.1307** | **0.0997** |
| BET | | **0.0797** | **0.1303** | **0.1753** | **0.1483** | | 0.0643 | 0.1039 | 0.1454 | 0.1204 | | 0.0511 | 0.0723 | 0.1294 | 0.0973 |
| $p$-value | | 0.0024 | 0.0001 | 0.0391 | 0.0012 | | 0.7815 | 0.2179 | 0.0514 | 0.0924 | | 0.1962 | 0.1337 | 0.2578 | 0.0414 |
| PEP | | 0.0594 | 0.0992 | 0.1274 | 0.1102 | | 0.0552 | 0.0914 | 0.1327 | 0.1081 | | 0.0414 | 0.0631 | 0.1121 | 0.0839 |
| ES | | 0.0646 | 0.1041 | 0.1477 | 0.1217 | | 0.0589 | 0.0935 | 0.1380 | 0.1115 | | 0.0413 | 0.0548 | 0.1112 | 0.0796 |
| MR | 90% | 0.0617 | 0.1013 | 0.1403 | 0.1171 | 90% | 0.0466 | 0.0764 | 0.1123 | 0.0911 | 90% | 0.0399 | 0.0536 | 0.1069 | 0.0769 |
| CIESS | | 0.0721 | 0.1193 | 0.1589 | 0.1353 | | 0.0601 | 0.0980 | 0.1380 | 0.1140 | | **0.0448** | 0.0613 | **0.1131** | 0.0841 |
| BET | | **0.0736** | **0.1212** | **0.1620** | **0.1376** | | **0.0623** | **0.1012** | **0.1426** | **0.1177** | | 0.0434 | **0.0648** | 0.1122 | **0.0845** |
| $p$-value | | 0.0165 | 0.0481 | 0.0091 | 0.0226 | | 0.0005 | 0.0205 | 0.0601 | 0.0232 | | 0.1454 | 0.0554 | 0.6608 | 0.0655 |
| PEP | | 0.0372 | 0.0666 | 0.0803 | 0.0712 | | 0.0247 | 0.0471 | 0.0674 | 0.0554 | | 0.0351 | 0.0591 | 0.0897 | 0.0712 |
| ES | | 0.0446 | 0.0727 | 0.1085 | 0.0872 | | 0.0527 | 0.0870 | **0.1213** | **0.1007** | | 0.0336 | 0.0478 | 0.0952 | 0.0682 |
| MR | 95% | 0.0500 | 0.0834 | 0.1178 | 0.0975 | 95% | 0.0438 | 0.0739 | 0.1040 | 0.0859 | 95% | 0.0329 | 0.0489 | 0.0918 | 0.0674 |
| CIESS | | 0.0513 | 0.0853 | 0.1214 | 0.0997 | | 0.0505 | 0.0856 | 0.1129 | 0.0964 | | 0.0361 | 0.0567 | 0.0933 | 0.0717 |
| BET | | **0.0627** | **0.1037** | **0.1374** | **0.1171** | | **0.0528** | **0.0883** | 0.1172 | 0.9980 | | **0.0388** | **0.0616** | **0.0963** | **0.0756** |
| $p$-value | | 0.0001 | 0.0002 | 0.0001 | 0.0001 | | 0.1896 | 0.0638 | 0.7269 | 0.3608 | | 0.0321 | 0.1107 | 0.1104 | 0.0010 |